



OPI AI Studio 配置记录 - Windows（驱动+CANN+llama.cpp）

这篇文章记录在 Windows 环境下配置 OPI AI Studio 的完整流程，包括测试模式设置、驱动安装、CANN 环境变量配置，以及用 `llama-cli.exe` 进行大模型推理。

一、环境与准备

开始前建议先确认以下信息：

1. 系统为 Windows。
2. 驱动安装包、CANN 压缩包、`llama-cli.exe` 与模型文件均可从以下地址下载：
<https://pan.baidu.com/s/1Gik8ghnBnZBIUUkAlaM8Tg?pwd=psjb>
3. 使用产品附带 USB4 线缆连接设备。

系统信息参考：

处理器

Intel(R) Core(TM) Ultra 9 285H

2.90 GHz

机带 RAM

64.0 GB

速度: 5600 MT/秒

图形卡

128 MB

Intel(R) Arc(TM) 140T GPU (32GB)

存储

954 GB

已使用 954 GB 中的 82 GB

DESKTOP-6A40LHA

雷神MIX

重命名这台电脑

设备信息

复制

^

设备名称	DESKTOP-6A40LHA
处理器	Intel(R) Core(TM) Ultra 9 285H (2.90 GHz)
机带 RAM	64.0 GB (63.5 GB 可用)
图形卡	Intel(R) Arc(TM) 140T GPU (32GB) (128 MB)
存储	已使用 954 GB 中的 82 GB
设备 ID	73C3B602-7BBB-4C42-986A-9B9F7E4829C7
产品 ID	00342-32133-11770-AAOEM
系统类型	64 位操作系统, 基于 x64 的处理器
笔和触控	没有可用于此显示器的笔或触控输入

二、将系统切换到测试模式

1. 右键开始菜单，打开 终端（管理员） 或 Windows PowerShell（管理员）。
2. 输入命令：

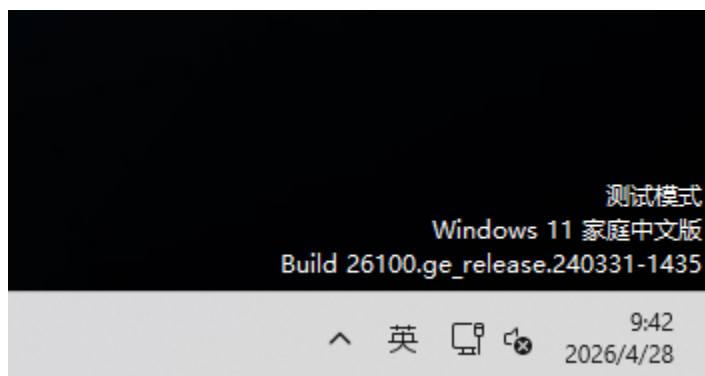
```
bcdedit /set testsigning on
```

3. 出现“操作成功”后，重启系统。
4. 重启后桌面右下角应出现“测试模式”水印。

命令执行示意：

```
PS C:\Users\never> bcdedit /set testsigning on  
操作成功完成。
```

测试模式水印示意：



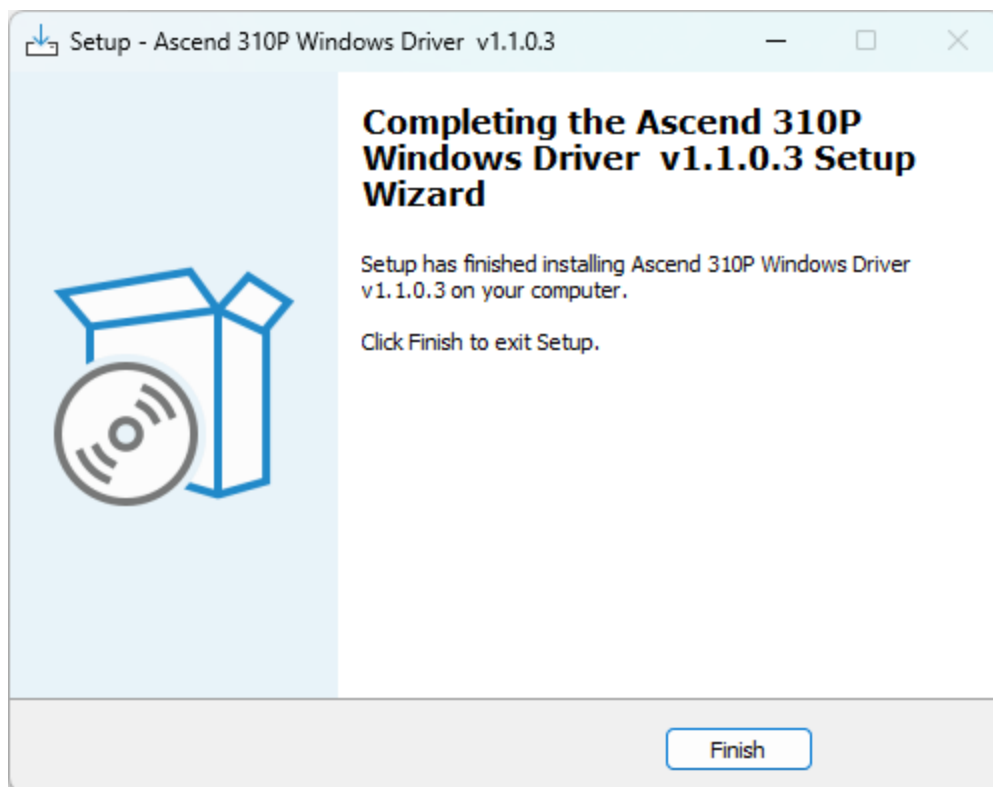
三、安装驱动并验证设备识别

1. 安装驱动 Ascend310P_1.1.0.3.exe。
2. 重启系统。
3. 启动 OPi AI Studio，并使用 USB4 线缆连接电脑。

注意事项：

先启动 OPi AI Studio 并连接到主机，再启动主机。若顺序错误，可能出现设备无法识别。

驱动安装界面示意：



4. 打开管理员终端，执行：

```
npu-smi info
```

5. 若终端能正确打印 OPi AI Studio 相关信息，则驱动安装成功。

识别成功示意：

```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

PS C:\Users\never> npu-smi info
[2026/04/28 10:05:49][1656][never][localhost][dcmi_mcu_base_inf.c,dcmi_passthru_mcu,527]:opcode = 0xf, ms_rsp->opcode = 0xf, errorcode=1.
[2026/04/28 10:05:49][1656][never][localhost][dcmi_mcu_base_inf.c,dcmi_mcu_get_info_by_npu,696]:call dcmi_passthru_mcu failed. err is -8005.
[2026/04/28 10:05:49][1656][never][localhost][dcmi_mcu_base_inf.c,dcmi_mcu_get_info,791]:call dcmi_mcu_get_info_inn failed. err is -8005.
[2026/04/28 10:05:49][1656][never][localhost][dcmi_mcu_base_inf.c,dcmi_mcu_get_fix_word,364]:call dcmi_mcu_get_info_fix failed. ret is -8005.
[2026/04/28 10:05:49][1656][never][localhost][dcmi_mcu_operate.c,dcmi_mcu_get_board_info,344]:get mcu board id fail.ret is -8005
[2026/04/28 10:05:49][1656][never][localhost][dcmi_init.c,dcmi_flush_card_info_board_id_pos,663]:dcmi_mcu_get_board_info failed. err is -8005

+-----+
| npu-smi 25.0.0.1 | Version: 25.0.0.1 |
+-----+
| NPU   Name      | Health          | Power(W)   Temp(C)     Hugespages-Usage(page) |
| Chip  Device    | Bus-Id         | AICore(%)  Memory-Usage(MB)       |
+-----+
| 179200 310P1    | OK              | 0.0        39          NA / NA               |
| 0      0        | 0000:04:00:0   | 0          1791 / 89575       |
+-----+
| NPU   Chip      | Process id      | Process name | Process memory(MB)    |
+-----+
| No running processes found in NPU 179200 |
+-----+
```

若安装成功后依旧报错如下，可以尝试重启整套环境：

```
PS C:\Users\never> npu-smi info
[2026/04/28 09:56:27][11180][never][localhost][dcmi_npu_operate.c,dcmi_get_npu_device_list,2199]:dsmi_get_device_count failed. err is 4.

[2026/04/28 09:56:27][11180][never][localhost][dcmi_api.c,dcmi_board_init,53]:dcmi_get_device_list failed. err is -8001.

[2026/04/28 09:56:27][11180][never][localhost][dcmi_api.c,dcmi_init,100]:dcmi_board_init failed. err is -8001

dcmi module initialize failed. ret is -8001
```

四、安装 CANN 包

1. 解压 CANN8.0.0.zip。
2. 将解压后的目录放到：

C:\Program Files\CANN8.0.0

3. 手动创建目录：

C:\Program Files\Huawei\Ascend

五、配置环境变量

1. 在开始菜单搜索“环境变量”，打开“编辑系统环境变量”。
2. 在“系统变量”中新建以下变量：

```
ASCEND_HOME_PATH = C:\Program Files\CANN8.0.0  
ASCEND_OPP_PATH  = C:\Program Files\CANN8.0.0
```

3. 编辑系统变量 `Path`，新增：

```
C:\Program Files\CANN8.0.0
```

环境变量配置示意：

编辑环境变量



%USERPROFILE%\AppData\Local\Microsoft\WindowsApps
C:\Program Files\CANN8.0.0

新建(N)

编辑(E)

浏览(B)...

删除(D)

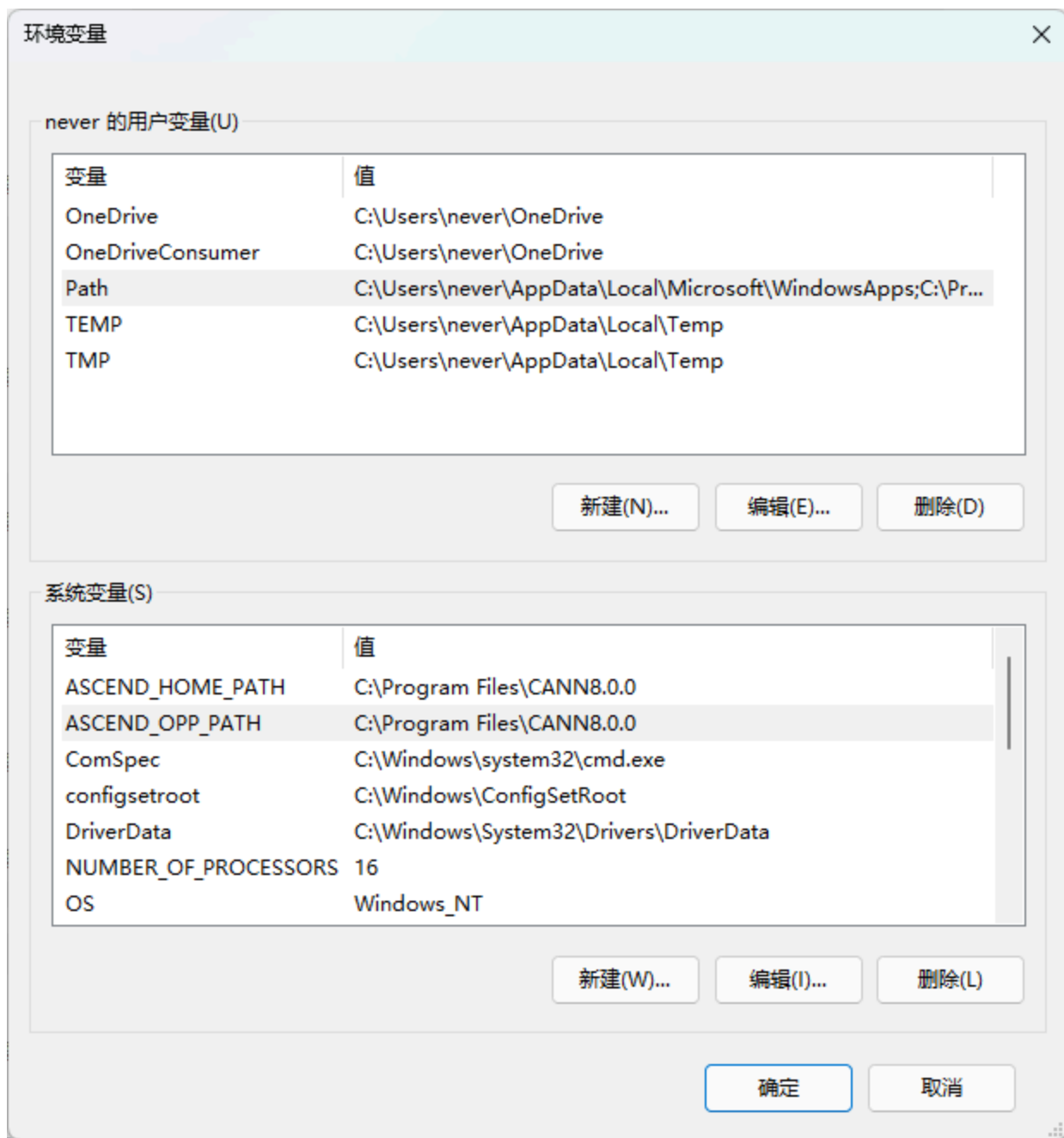
上移(U)

下移(O)

编辑文本(T)...

确定

取消



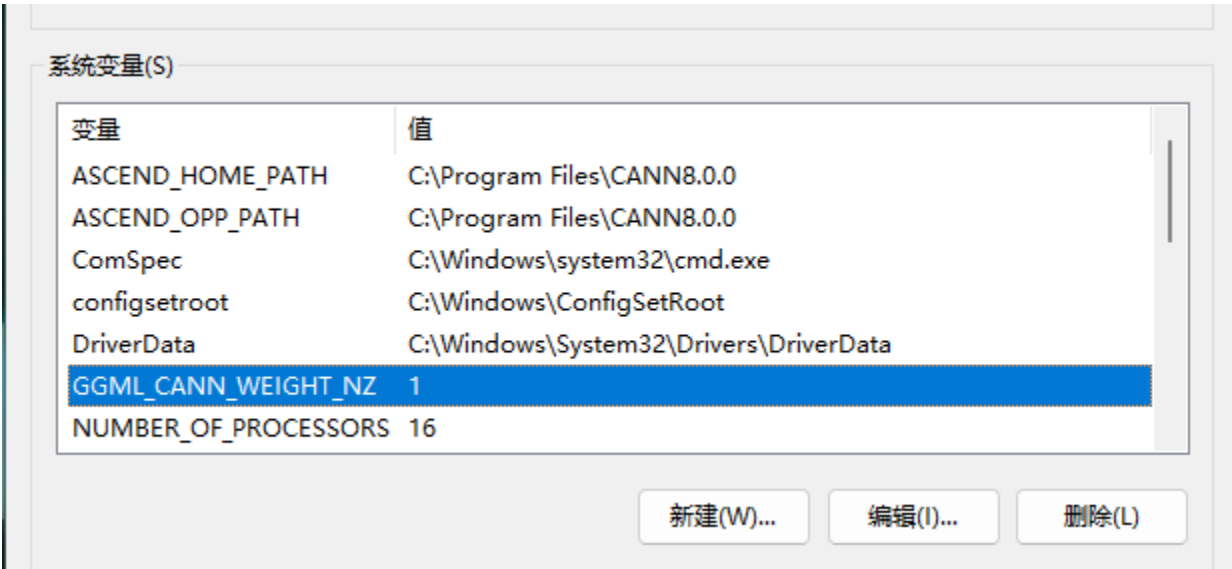
六、配置 llama.cpp 并运行模型

1) 添加 llama.cpp 相关环境变量

在系统环境变量中新增：

```
GGML_CANN_WEIGHT_NZ = 1
```

配置示意：



2) 启动推理

打开管理员终端，执行：

```
{llama-cli.exe 文件路径} -m {模型文件路径}
```

命令行调用示意：


```

PS C:\Users\never> C:\Users\never\Desktop\llama.cpp\llama-cli.exe -m C:\Users\never\Desktop\models\qwen2.5-0.5b-instruct-fp16.gguf
register_backend: registered backend CANN (1 devices)
register_device: registered device CANN0 (Ascend310P1)
register_backend: registered backend CPU (1 devices)
register_device: registered device CPU (Intel(R) Core(TM) Ultra 9 285H)
load_backend: failed to find ggml_backend_init in C:\Users\never\Desktop\llama.cpp\ggml-cann.dll
load_backend: failed to find ggml_backend_init in C:\Users\never\Desktop\llama.cpp\ggml-cpu.dll
build: 0 (unknown) with MSVC 19.29.30136.0 for x64 (debug)
main: llama backend init
main: load the model and apply lora adapter, if any
llama_model_load_from_file_impl: using device CANN0 (Ascend310P1) - 88320 MiB free
llama_model_loader: loaded meta data with 26 key-value pairs and 291 tensors from C:\Users\never\Desktop\models\qwen2.5-0.5b-instruct-fp16.gguf (version GGUF V3 (latest))
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0:                general.architecture str      = qwen2
llama_model_loader: - kv 1:                general.type str           = model
llama_model_loader: - kv 2:                general.name str           = qwen2.5-0.5b-instruct
llama_model_loader: - kv 3:                general.version str        = v0.1
llama_model_loader: - kv 4:                general.finetune str       = qwen2.5-0.5b-instruct
llama_model_loader: - kv 5:                general.size_label str     = 630M
llama_model_loader: - kv 6:                qwen2.block_count u32      = 24
llama_model_loader: - kv 7:                qwen2.context_length u32   = 8192
llama_model_loader: - kv 8:                qwen2.embedding_length u32 = 896
llama_model_loader: - kv 9:                qwen2.feed_forward_length u32 = 4864
llama_model_loader: - kv 10:               qwen2.attention.head_count u32 = 16
llama_model_loader: - kv 11:               qwen2.attention.head_count_kv u32 = 2
llama_model_loader: - kv 12:               qwen2.rope.freq_base f32    = 1000000.000000
llama_model_loader: - kv 13:               qwen2.attention.layer_norm_rms_epsilon f32 = 0.000001
llama_model_loader: - kv 14:               general.file_type u32      = 1
llama_model_loader: - kv 15:               tokenizer.ggml.model str    = gpt2
llama_model_loader: - kv 16:               tokenizer.ggml.pre str      = qwen2
llama_model_loader: - kv 17:               tokenizer.ggml.tokens arr[str,151936] = [" ", "'", '"', '##', '$', '%', 'c', '!', ...
llama_model_loader: - kv 18:               tokenizer.ggml.token_type arr[i32,151936] = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
llama_model_loader: - kv 19:               tokenizer.ggml.merges arr[str,151387] = ["' ", "G G", "i n", "G t",...
llama_model_loader: - kv 20:               tokenizer.ggml.eos_token_id u32    = 151645
llama_model_loader: - kv 21:               tokenizer.ggml.padding_token_id u32 = 151643
llama_model_loader: - kv 22:               tokenizer.ggml.bos_token_id u32    = 151643
llama_model_loader: - kv 23:               tokenizer.ggml.add_bos_token bool  = false
llama_model_loader: - kv 24:               tokenizer.chat_template str     = {% if tools %}\n  {{- '<|im_start|>...'
llama_model_loader: - kv 25:               general.quantization_version u32   = 2
llama_model_loader: - type f32: 121 tensors
llama_model_loader: - type f16: 170 tensors
print_info: file format = GGUF V3 (latest)
print_info: file type   = F16
print_info: file size   = 1.17 GiB (16.00 BPW)
load: special tokens cache size = 22
load: token to piece cache size = 0.9310 MB

```

当终端出现 `>` 且光标闪烁时，表示模型已成功启动：

```

管理: Windows PowerShell
llama_context: causal_attn = 1
llama_context: flash_attn = 0
llama_context: freq_base = 1000000.0
llama_context: freq_scale = 1
llama_context: n_ctx_per_seq (4096) < n_ctx_train (8192) -- the full capacity of the model will not be utilized
ggml_backend_cann_context: device 0 async operator submission is OFF
llama_context: CANN_Host output buffer size = 0.58 MiB
llama_kv_cache_unified: CANN0 KV buffer size = 48.00 MiB
llama_kv_cache_unified: size = 48.00 MiB ( 4096 cells, 24 layers, 1 seqs), K (f16): 24.00 MiB, V (f16): 24.00 MiB
llama_context: CANN0 compute buffer size = 298.50 MiB
llama_context: CANN_Host compute buffer size = 9.76 MiB
llama_context: graph nodes = 942
llama_context: graph splits = 2
common_init_from_params: setting dry_penalty_last_n to ctx_size = 4096
common_init_from_params: warming up the model with an empty run - please wait ... (--no-warmup to disable)
new_pool_for_device: device 0 use vmm pool
main: llama threadpool init, n_threads = 16
main: chat template is available, enabling conversation mode (disable it with --no-cnv)
main: chat template example:
<|im_start|>system
You are a helpful assistant<|im_end|>
<|im_start|>user
Hello<|im_end|>
<|im_start|>assistant
Hi there<|im_end|>
<|im_start|>user
How are you?<|im_end|>
<|im_start|>assistant

system_info: n_threads = 16 (n_threads_batch = 16) / 16 | CPU : SSE3 = 1 | SSSE3 = 1 | AVX = 1 | AVX2 = 1 | F16C = 1 | FMA = 1 | LLAMAFILE = 1 | OPENMP = 1 | REPACK = 1 |

main: interactive mode on.
sampler seed: 2385472989
sampler params:
  repeat_last_n = 64, repeat_penalty = 1.000, frequency_penalty = 0.000, presence_penalty = 0.000
  dry_multiplier = 0.000, dry_base = 1.750, dry_allowed_length = 2, dry_penalty_last_n = 4096
  top_k = 40, top_p = 0.950, min_p = 0.050, xtc_probability = 0.000, xtc_threshold = 0.100, typical_p = 1.000, top_n_sigma = -1.000, temp = 0.800
  mirostat = 0, mirostat_lr = 0.100, mirostat_ent = 5.000
sampler chain: logits -> logit-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
generate: n_ctx = 4096, n_batch = 2048, n_predict = -1, n_keep = 0

== Running in interactive mode. ==
- Press Ctrl+C to interject at any time.
- Press Return to return control to the AI.
- To return control without starting a new line, end your input with '/'.
- If you want to submit another line, end your input with '\'.
- Not using system message. To change it, set a different value via -sys PROMPT

> |

```

3) 提问与结束

1. 在 `>` 后输入提问内容并回车。
2. 等待模型输出完成。
3. 当再次出现 `>` 且光标闪烁时，按 `Ctrl + C` 结束推理。

推理输出示意：

```
emp-ext -> dist
generate: n_ctx = 4096, n_batch = 2048, n_predict = -1, n_keep = 0

== Running in interactive mode. ==
- Press Ctrl+C to interject at any time.
- Press Return to return control to the AI.
- To return control without starting a new line, end your input with '/'.
- If you want to submit another line, end your input with '\'.
- Not using system message. To change it, set a different value via -sys PROMPT

> 问:在深度学习领域, 卷积神经网络(CNN)和循环神经网络(RNN)在处理图像和时间序列数据方面有哪些关键差异?请详细解释每种网络结构的主要特点, 包括它们在不同类型的任务中如何应用, 例如图像识别、自然语言处理和时间序列预测。此外, 讨论一下这些网络如何处理过拟合问题, 以及如何使用正则化技术如dropout来提高模型的泛化能力。最后, 探讨一下在当前的人工智能研究中, 这些网络如何与其他模型如 Transformer 结合, 以解决复杂的机器学习问题, 并给出些这些模型在实际应用中的成功案例。
在深度学习领域, 卷积神经网络(CNN)和循环神经网络(RNN)都是重要的网络结构, 它们各自在处理图像和时间序列数据方面有着不同的特点和应用。

1. 卷积神经网络(CNN):
   卷积神经[ERROR](13976)D:\cann\0605_release\abl\slog\liblog\slog\plog\plog_file_mgr.c:218: fsync fail, ret=-1, strerr=
Input/output error.
   网络(CNN)是一种结构复杂, 学习能力强, 适用于处理图像数据的模型。卷积神经网络(CNN)的特征在于它的卷积层, 卷积层可以对图像中的每个像素进行处理, 通过计算滤波器的输出来[ERROR](13976)D:\cann\0605_release\abl\slog\liblog\slog\plog\plog_file_mgr.c:218: fsync fail, ret=-1, strerr=Input/output error.
   获得图像的特征。卷积层通过将输入图像进行分层, 然后在每一层上计算滤波器的输出来获得特征, 从而可以学习到图像中的特征。

   卷积神经网络(CNN)的主要特点包括:
   - ◆[ERROR](13976)D:\cann\0605_release\abl\slog\liblog\slog\plog\plog_file_mgr.c:218: fsync fail, ret=-1, strerr=Input/output error.
   ◆积层可以学习到图像中的特征
```

已知日志说明：

在我的推理输出中可以看到类似以下日志：

```
[ERROR](9836)D:\cann\0605_release\abl\slog\liblog\slog\plog\plog_file_mgr.c:218: fsync fa
```

这个报错大概率来自 CANN 的 `slog` 日志模块。该模块主要按 Linux 服务器环境设计，Windows 文件系统对 `fsync` 语义支持与 Linux 存在差异，在特定条件下会触发 `fsync fail`。当前看属于日志写盘路径的兼容性问题，通常表现为持续报错日志，但推理进程本身并不崩溃。

结束后会看到性能统计信息（包含平均 `tps`）：

```
> llama_perf_sampler_print: sampling time = 312.24 ms / 870 runs ( 0.36 ms per token, 2786.36 tokens per second)
llama_perf_context_print: load time = 3873.65 ms
llama_perf_context_print: prompt eval time = 84.67 ms / 142 tokens ( 0.60 ms per token, 1677.14 tokens per second)
llama_perf_context_print: eval time = 63699.65 ms / 727 runs ( 87.62 ms per token, 11.41 tokens per second)
llama_perf_context_print: total time = 923900.39 ms / 869 tokens
Interrupted by user
```

七、常见问题

1) 设备无法识别

优先检查启动顺序与 USB4 连接状态：

1. 先启动 OPi AI Studio 并连接主机。
2. 再启动主机。
3. 重新执行 `npu-smi info` 验证。

2) `fsync fail` 日志反复出现

典型日志如下：

```
[ERROR] (9836)D:\cann\0605_release\abl\slog\liblog\slog\plog\plog_file_mgr.c:218: fsync fa
```

可能原因：

1. CANN 的 `slog` 在 Linux 场景设计较完善。
2. Windows 文件系统对 `fsync` 语义支持不完全一致。
3. CANN 在 Windows 路径上未完全做容错，导致该日志持续出现。

八、总结

目前在 Windows 环境下，OPi AI Studio 整体上已经完成适配，可以正常完成推理任务：

1. 驱动识别正常（`npu-smi info` 可读）。
2. CANN 环境变量可用。
3. `llama-cli` 可以成功加载模型并启动推理。

但由于前面提到的 `fsync fail` 报错问题，推理输出的可视性会受到影响，结果显示常常断断续续，不够稳定直观。

因此，后续将改用 Ubuntu 环境重新配置，并继续验证推理链路的稳定性与输出效果。